

Machine Translation

Machine Translation

- **The task:** given a string in one language (source) and a target language, generate a translation for it in the target language
- Variants of the task:
 - Scope of the text: sentence/paragraph/document
 - Single output or space of possible outputs

Ambiguity Resolution

- Syntactic and semantic ambiguities must be properly resolved for correct translation:
 - John plays the guitar → ג'ון מנגן בגיטרה
 - John plays soccer → ג'ון משחק בכדורגל
 - בניין גבוה → tall building
 - ציפור עפה גבוה → a bird flies high
- Jests: (when applying English → Russian → English)
 - “The spirit is willing but the flesh is weak.” ⇒ “The liquor is good but the meat is spoiled”
 - “Out of sight, out of mind.” ⇒ “Invisible idiot”

Translation Quality

- Achieving literary quality translation is very difficult
- Existing MT systems can generate rough translations that frequently at least convey the **gist** of a document
- High quality translations possible when specialized to narrow domains
- Some MT systems used in **computer-aided translation** in which a bilingual human post-edits the output to produce more readable accurate translations
- Frequently used to aid **localization** of software interfaces and documentation to adapt them to other languages

Linguistic Differences between Languages

- Syntactic variation between **SVO** (e.g. English), **SOV** (e.g. Hindi), and **VSO** (e.g. Arabic) languages
 - SVO languages tend to use prepositions
 - SOV languages tend to use postpositions
- **Pro-drop** languages regularly omit subjects that must be inferred when translated to other languages
 - I don't know. Did you like it? 知らない。気に入った？
Shiranai. Ki ni itta?
 know-NEGATIVE. like-PAST?
 - הלכתי הביתה I went home

Reminder: “Word” Order Differences

- Compare English with Japanese glosses:

“IBM bought Lotus”



“IBM Lotus bought”

“Sources said that IBM bought Lotus yesterday”

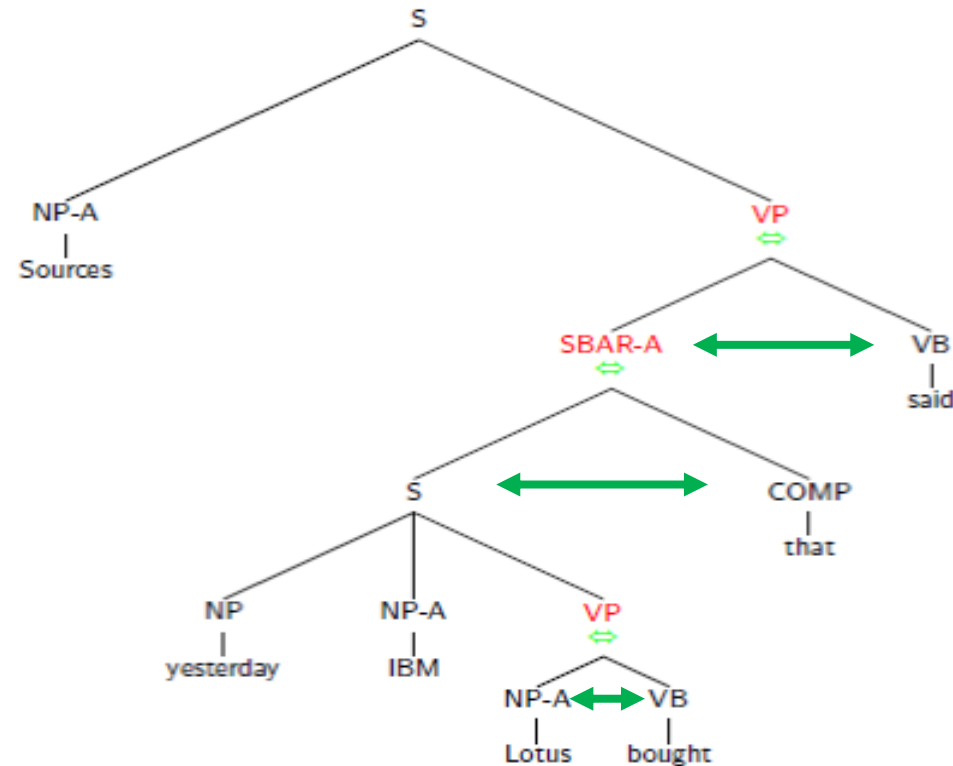


“Sources yesterday IBM Lotus bought that said”

- It is difficult to phrase the possible permutations in terms of words, easier in terms of phrases

Word Order Differences

- This can be seen by comparing English and Japanese Constituency Structures:

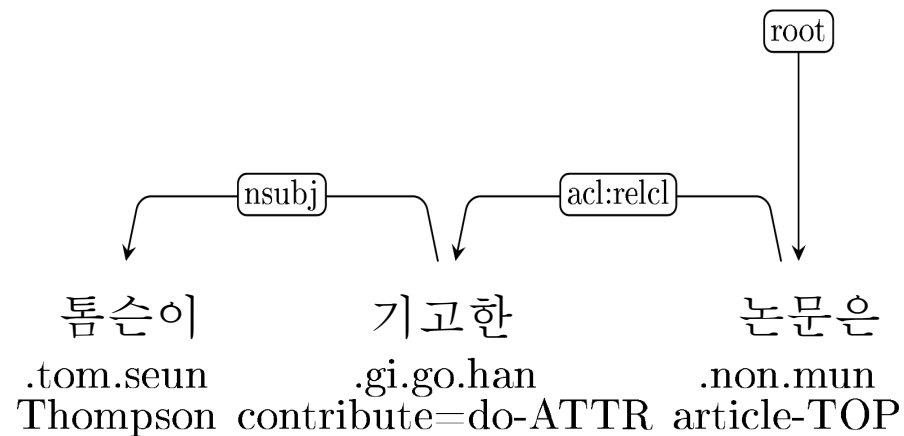
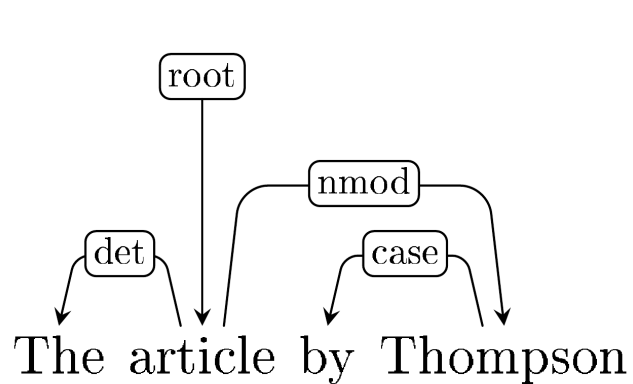


Linguistic Divergences

- Changes between some languages can be much more substantial

- Linguistic divergences (which are not word order differences):

- “I like him” ↔ “הוא מוצא חן בעיניי”
- “to be cold” ↔ “avoir froid” (lit. “to have cold”)



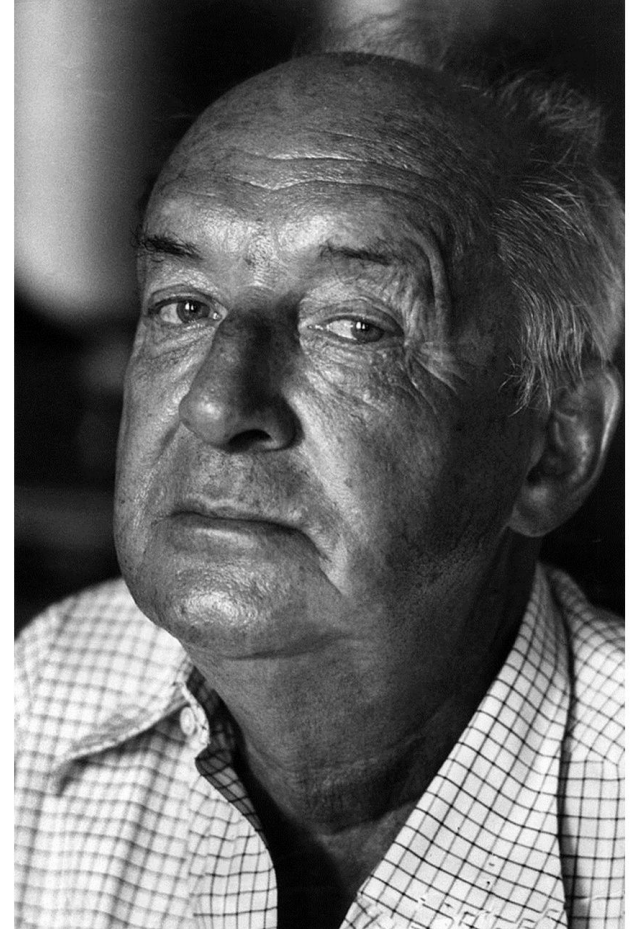
Lexical Gaps

- Some words in one language do not have a corresponding term in the other
 - *Siblings* in Hebrew?
 - *Fleuve* (river that flows into ocean) and *Rivière* (river that does not flow into ocean) in French
 - *Zechprellerei* (sometimes called 'dine and dash')

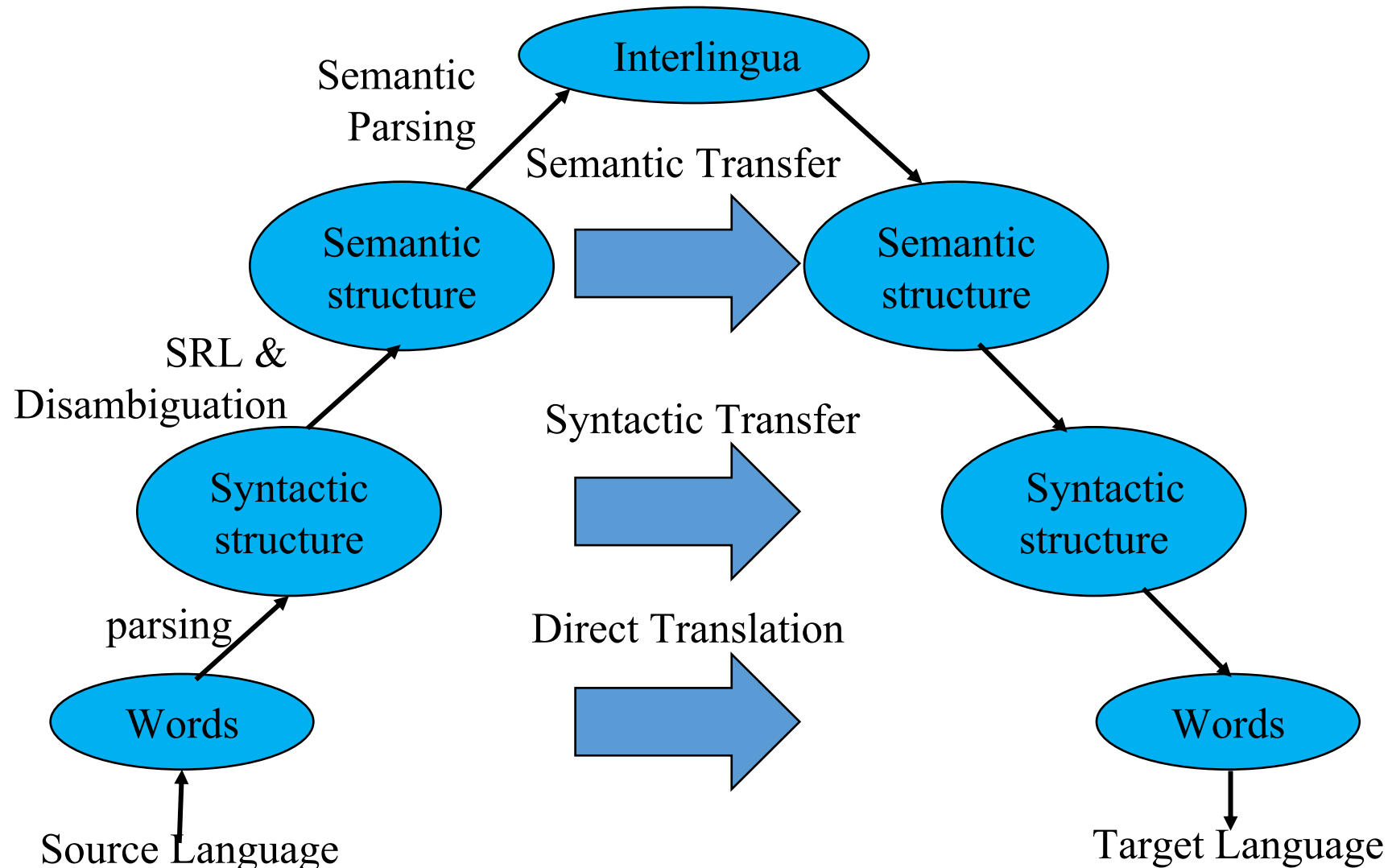


Lexical Gaps: Near Correspondents

"No single word in English renders all the shades of *toska*. At its deepest and most painful, it is a sensation of great spiritual anguish, often without any specific cause. At less morbid levels it is a dull ache of the soul, a longing with nothing to long for, a sick pining, a vague restlessness, mental throes, yearning. In particular cases it may be the desire for somebody of something specific, nostalgia, love-sickness. At the lowest level it grades into ennui, boredom." (Vladimir Nabokov)



Vauquois Triangle



Direct Transfer: Classic Approach

1. Morphological analysis as pre-processing:
 - “Mary didn’t slap the green witch” → “Mary DO:PAST not slap the green witch”
 - "מרי לא סטרה למכשפה הירוקה" →
"מרי לא סטר:עבר, נקבה ל מכשפה ה ירוק:נקבה"
2. Lexical Transfer
 - Mary DO:PAST not slap the green witch →
Maria no dar:PAST una bofetada a la verde bruja
3. Lexical Reordering
 - Maria no dar:PAST una bofetada a la bruja verde
4. Morphological generation
 - Maria no dió una bofetada a la bruja verde

Syntactic Transfer: Classic Approach

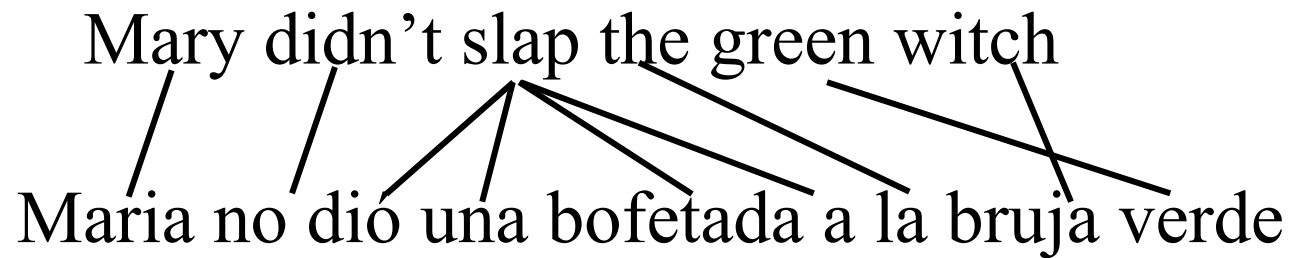
- Simple lexical reordering does not adequately handle more dramatic reordering such as that required to translate from an SVO to an SOV language.
- Need syntactic transfer rules that map parse tree for one language into one for another:
 - English to Spanish:
 - $NP \rightarrow Adj\ Nom \Rightarrow NP \rightarrow Nom\ ADJ$
 - English to Japanese:
 - $VP \rightarrow V\ NP \Rightarrow VP \rightarrow NP\ V$
 - $PP \rightarrow P\ NP \Rightarrow PP \rightarrow NP\ P$

Statistical Methods

- As with other areas of MT, manually crafted rules are low in coverage and don't scale well
- SMT acquires knowledge needed for translation from a **parallel corpus** or **bitext** that contains the same set of documents in two languages
 - With some models more supervision is required, such as a syntactic parser or a bi-lingual dictionary
- The *Canadian Hansards* (parliamentary proceedings in French and English) and the *Europarl* (protocols of the European Parliament, in all official EU languages) are well-known parallel corpora
 - Parallel corpora are also published through the annual conference for MT (WMT)

Word Alignment

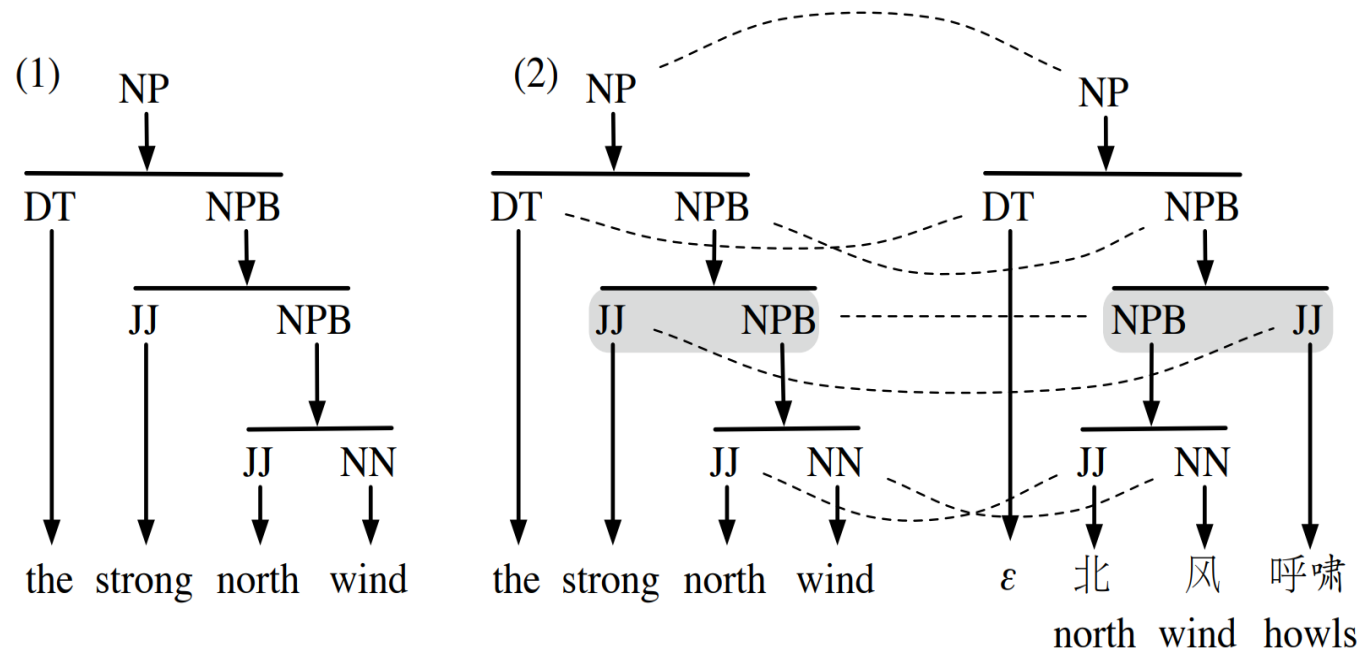
- One of the basic tools in MT
- Shows mapping between words in one language and the other



- Often done by finding co-occurrences across of words/phrases across parallel data

Syntax-Based Statistical Machine Translation

- Syntax-based approaches demonstrated improvement for translating between more distant language pairs, e.g., Chinese/English
- They assume that the syntactic trees of the derivations in the two languages are similar, except that each right-hand-side of a production may be permuted relative to the production in the other language
- These transformations are learned from syntax-annotated parallel data



Syntax-Based Statistical Machine Translation

- Despite the promise of such models, their assumption is still too weak to deal with all cross-linguistic divergences where the syntactic tree is entirely different

Jacky happened to meet the guy yesterday

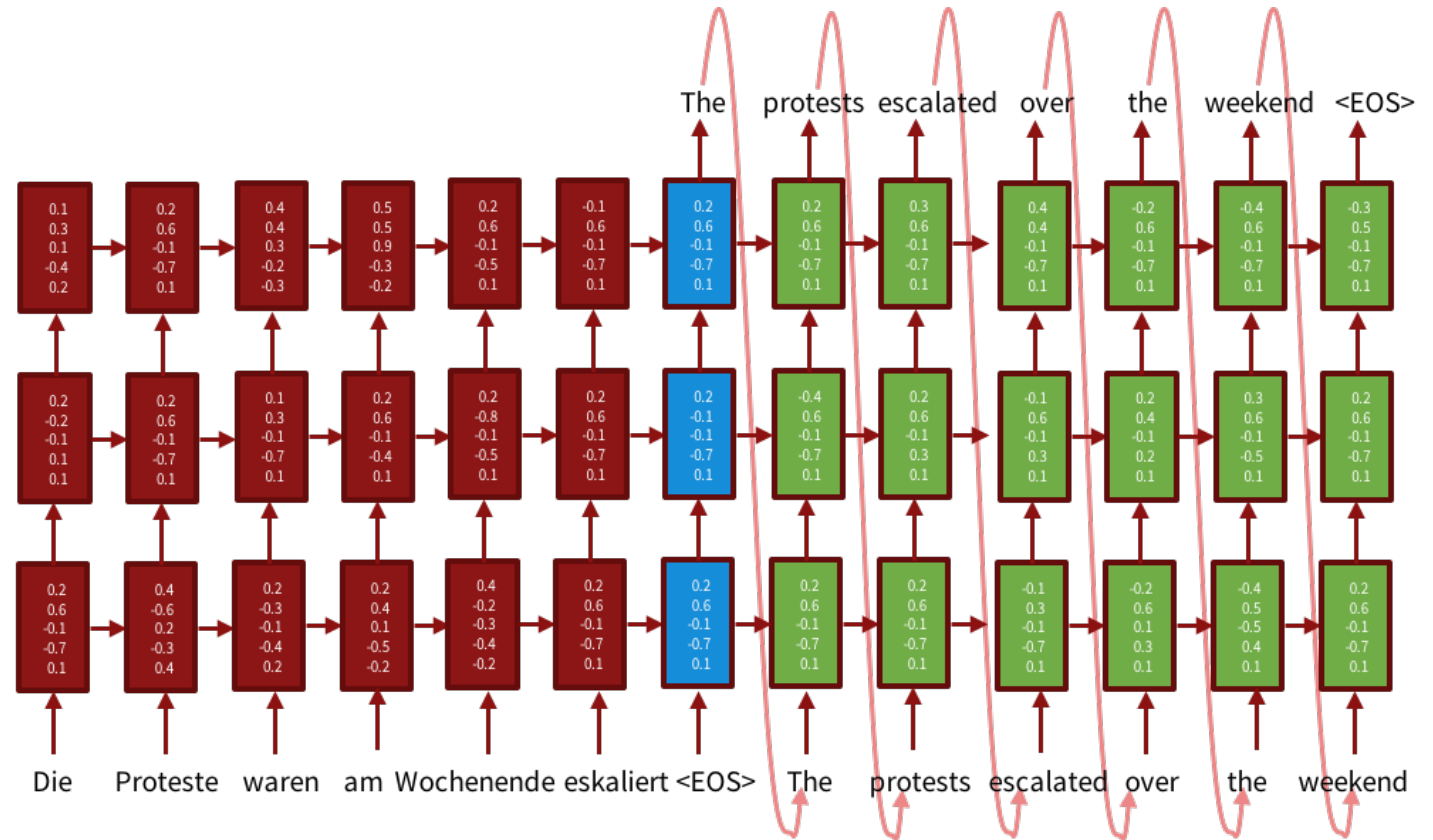
ג'קי פגש במקרה את הבחור אתמול

ג'קי קרה לפגוש את הבחור אתמול



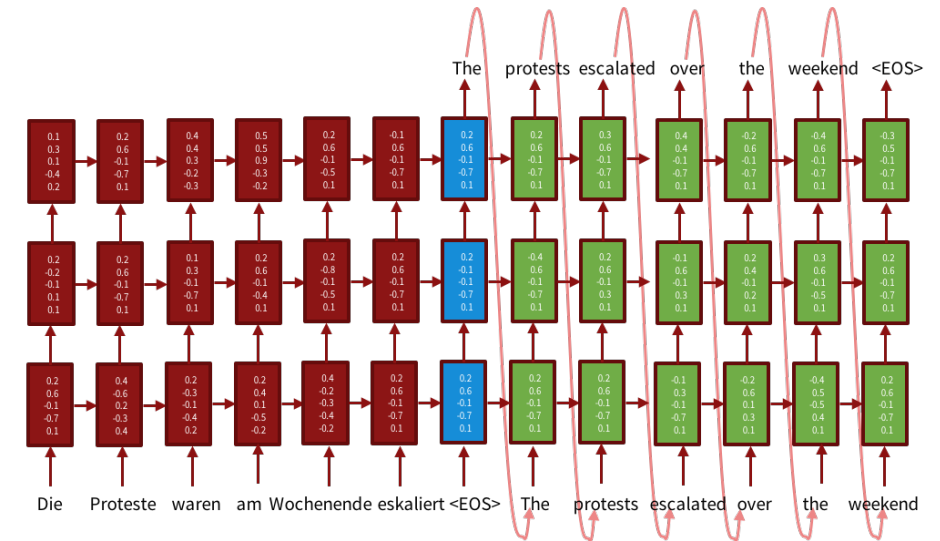
Neural Machine Translation

- The current state of the art outperforms phrase-based translation by a considerable margin in a variety of settings
- The standard architecture is called encoder-decoder



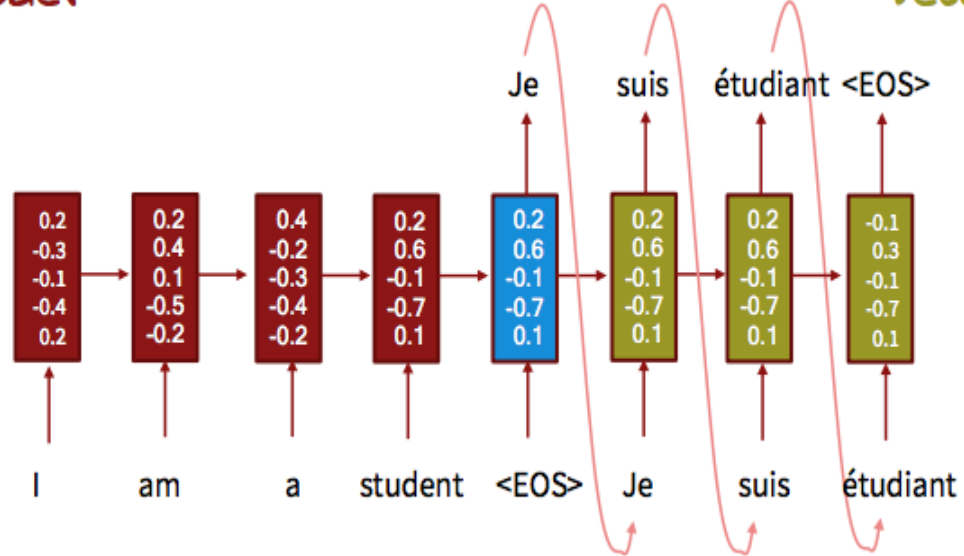
Neural Machine Translation

- Encoder-decoder systems have two stages:
 1. Encoding: the input sentence is encoded into a vector. This can be done by RNNs or other types of networks.
 2. Decoding: like a language model it assigns a probability to the next word. However, it also conditions on the encoded input.
- This is a neural LM, which is conditioned on the encoding of the input



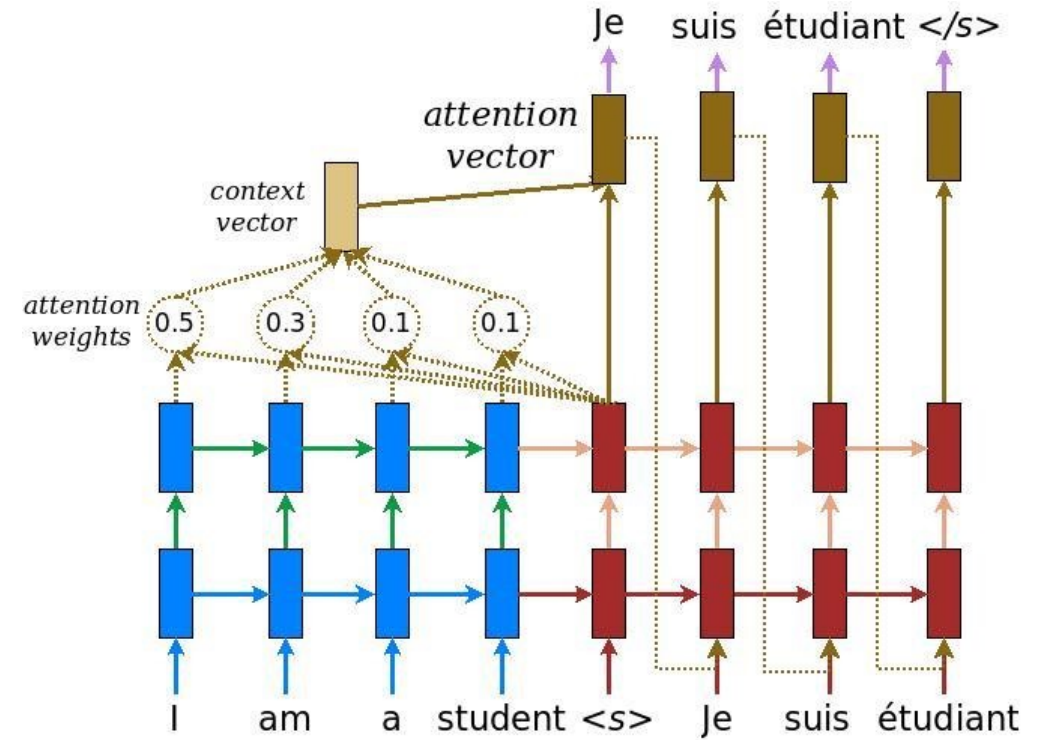
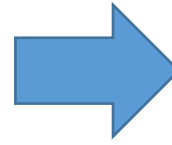
RNN+Attention

Encoder



$$h_t = \tanh(W[x_t] + Uh_{t-1} + b)$$

Decoder



RNN+Attention

- Attention is a mechanism for making specific parts of the input more accessible to the decoder
- Instead of the decoder only receiving the summary vector for the input (the last hidden state), using attention it receives a linear combination of the hidden states of the tokens of the sequence that are most relevant for the next decoded word
 - A form of a soft alignment
- The coefficients are themselves dynamically computed
 - We will define this formally in the next slides

Neural Machine Translation

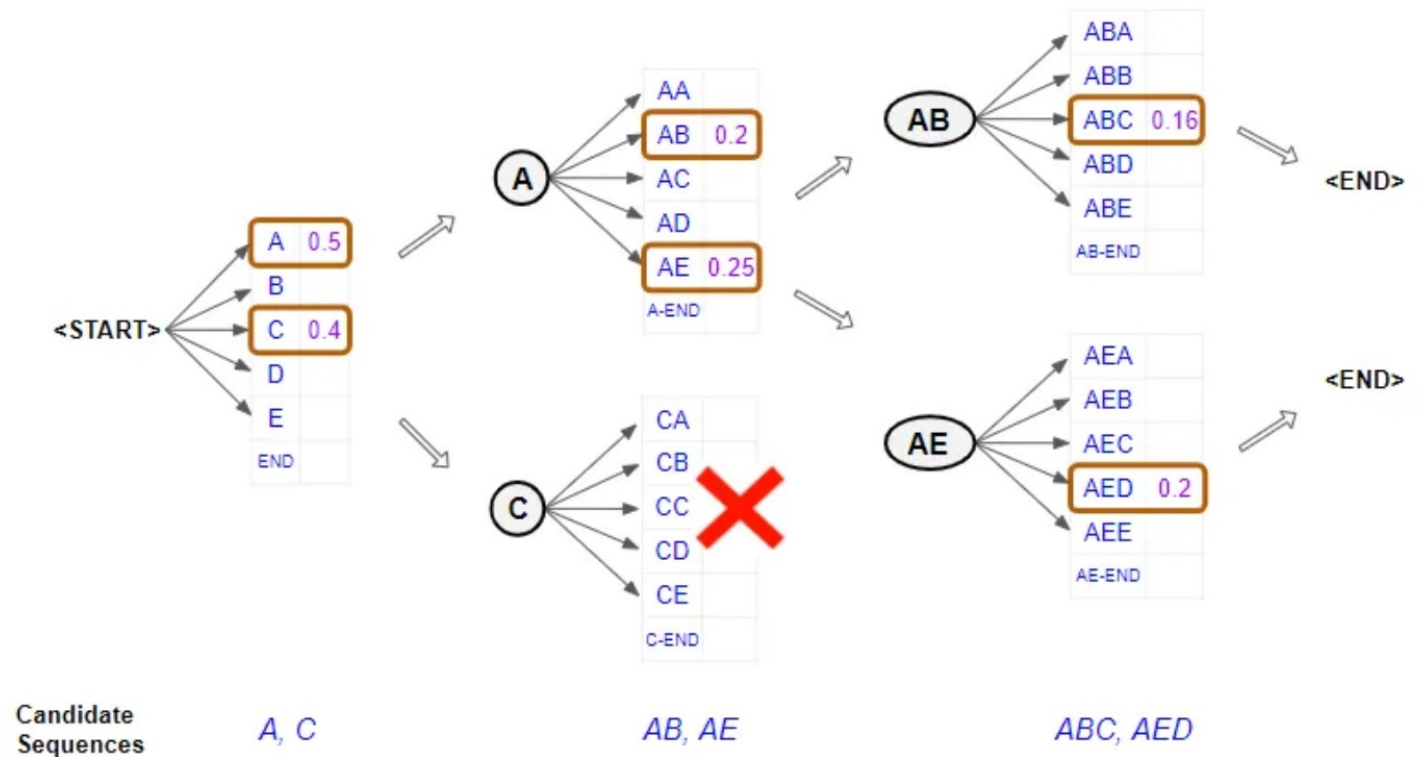
- Training: standard training is done by maximizing the average log probability of the next word, given the input and the prefix (y is the translation, x is the input sentence)

$$L(\theta) = \frac{1}{N} \sum_{i=1}^N \log[p_{\theta}(y_i | y_{i-1}, \dots, y_1, \mathbf{x})]$$

- This training procedure (teacher forcing) suffers from the problem of *exposure bias*: the model is only ever trained to predict the next token, given a gold standard prefix of a translation of $\mathbf{x}$

Neural Machine Translation

- Decoding:
 - Greedy decoding
 - Iteratively select the most likely token
 - Beam search



Neural Machine Translation: Beam Search

Input: $b > 0$ (beam size), P (probability of next token given prefix)

Terminal = []

$Q = \text{PriorityQueue}()$

$Q.\text{add}([\langle \text{START} \rangle], 1)$

while Q not empty:

$a, \text{value} = Q.\text{maximum}()$

 if a ends with $\langle \text{END} \rangle$:

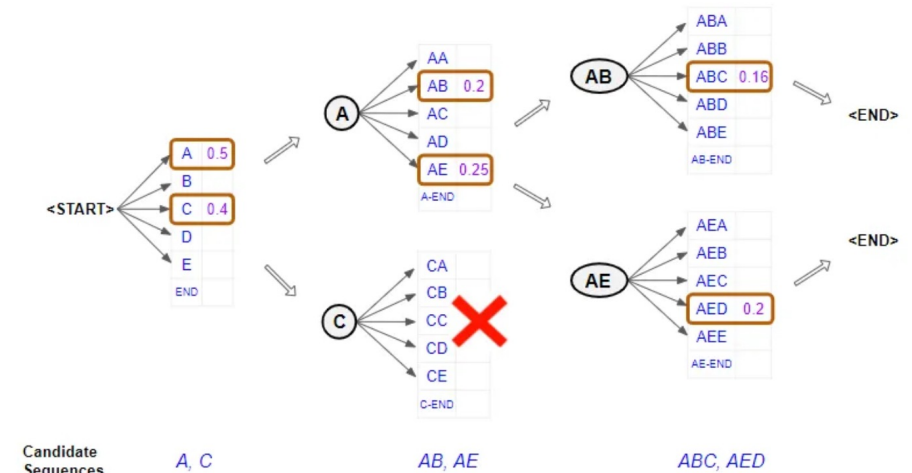
 Terminal.add($\langle a, \text{value} \rangle$)

 else:

$Q.\text{extend}([\langle a.\text{append}(v), \text{value} * P(v|a) \rangle \text{ for } v \text{ in Vocab}]$

$Q = \text{maximal } b \text{ elements in } Q$

return Terminal.maximum()



Exposure Bias

- The exposure bias stems from a discrepancy between training and decoding
 - During training - only gold standard prefixes
 - During decoding - computer generated prefixes
- Possible methods to address this:
 - Train on computer generated prefixes → but how will we know the correct next token then?
 - Use other methods, such as reinforcement learning that bypass the problem
 - Different loss functions

The Transformer Architecture

Attention and Transformers

- Attention: a way to model an implicit soft alignment between the words in the source sentence and the words in the target
 - Helps “select” what words in the source are most important for generating the next word
 - Inspired by human “attention:” limited in its capacity
- More generally, the same mechanism can be used for defining context as a distribution over tokens
 - We can also think of the previous layer as context for the next layer: this is called ‘self-attention’

Self-Attention

- Self-attention is the use of attention to generate, from a sequence of word embeddings, a new sequence of word embeddings whose representation is some aggregation of the word embeddings in the previous layer
- Why would we want to do that?

Attention Visualizations

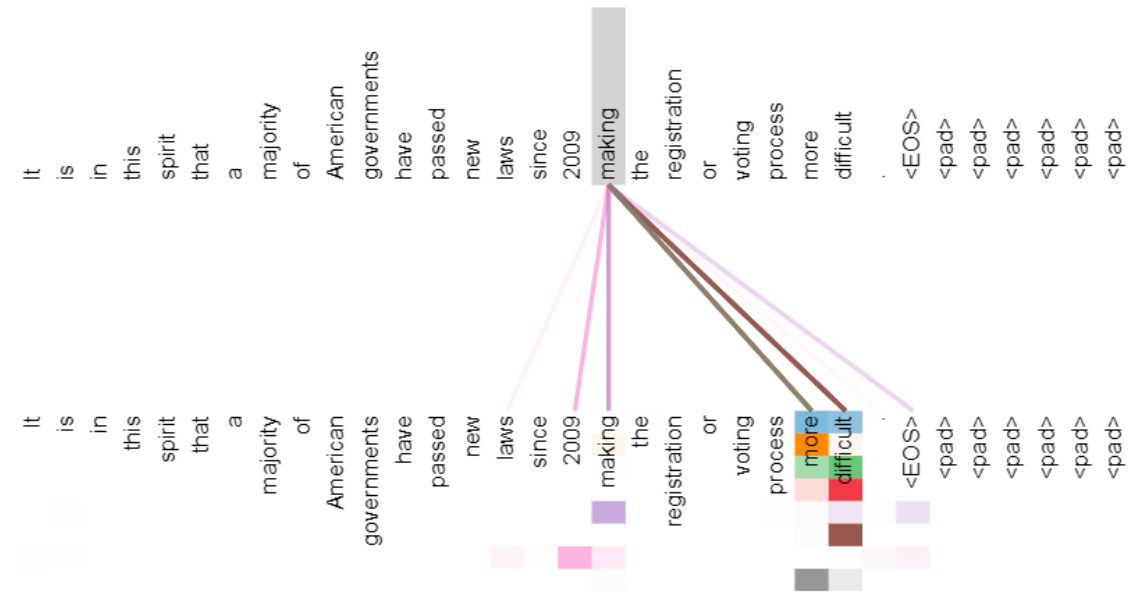
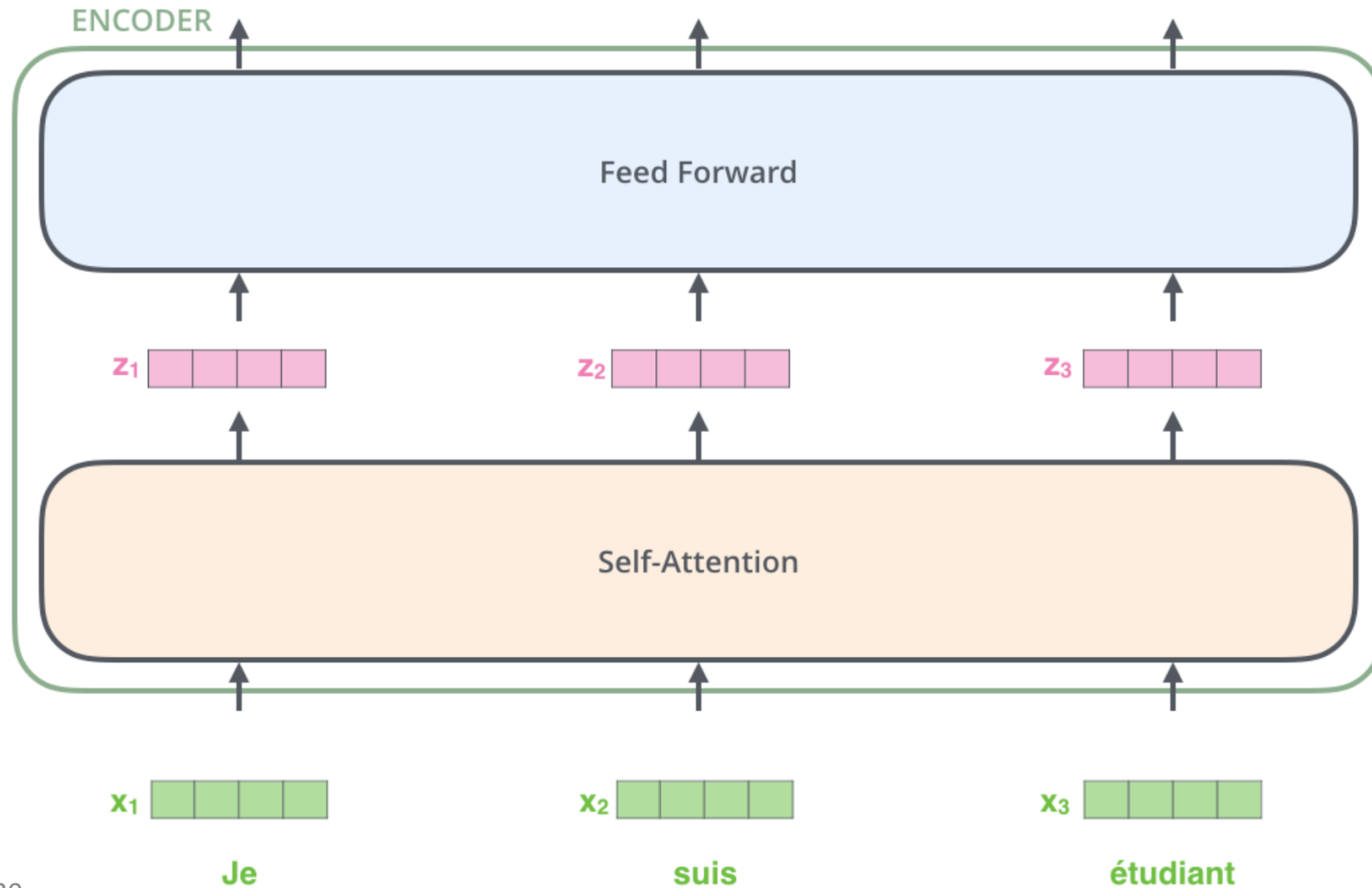


Figure 3: An example of the attention mechanism following long-distance dependencies in the encoder self-attention in layer 5 of 6. Many of the attention heads attend to a distant dependency of the verb 'making', completing the phrase 'making...more difficult'. Attentions here shown only for the word 'making'. Different colors represent different heads. Best viewed in color.

Self-Attention

- Traditional ('non-contextualized') embeddings of a word represent the word form over all its different senses
 - For example, the word *make* could be used in many different senses (e.g., להצחיק — make ... laugh, להקשות — make difficult, להבין — make a cake, make a dog's breakfast of ... - לעשות משהו ברישול)
 - The context of a word may disambiguate it, resulting in a representation that is tailored to the sentence it appears in
- Self-attention begins with embedding every word according to its form, and an embedding of its position
- It then iteratively constructs a representation of the word by self-attending to the representations in the previous layer

The Self-Attention Mechanism



The Illustrated Transformer

<https://jalammar.github.io/illustrated-transformer/>
[Jay Alammar](#)

The Illustrated Transformer

<https://jalammar.github.io/illustrated-transformer/>

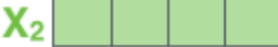
[Jay Alammar](#)

Input

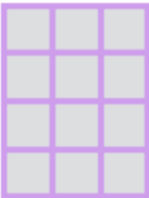
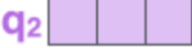
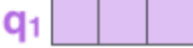
Thinking

Machines

Embedding

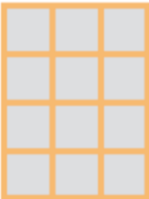
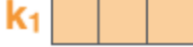


Queries



W^Q

Keys



W^K

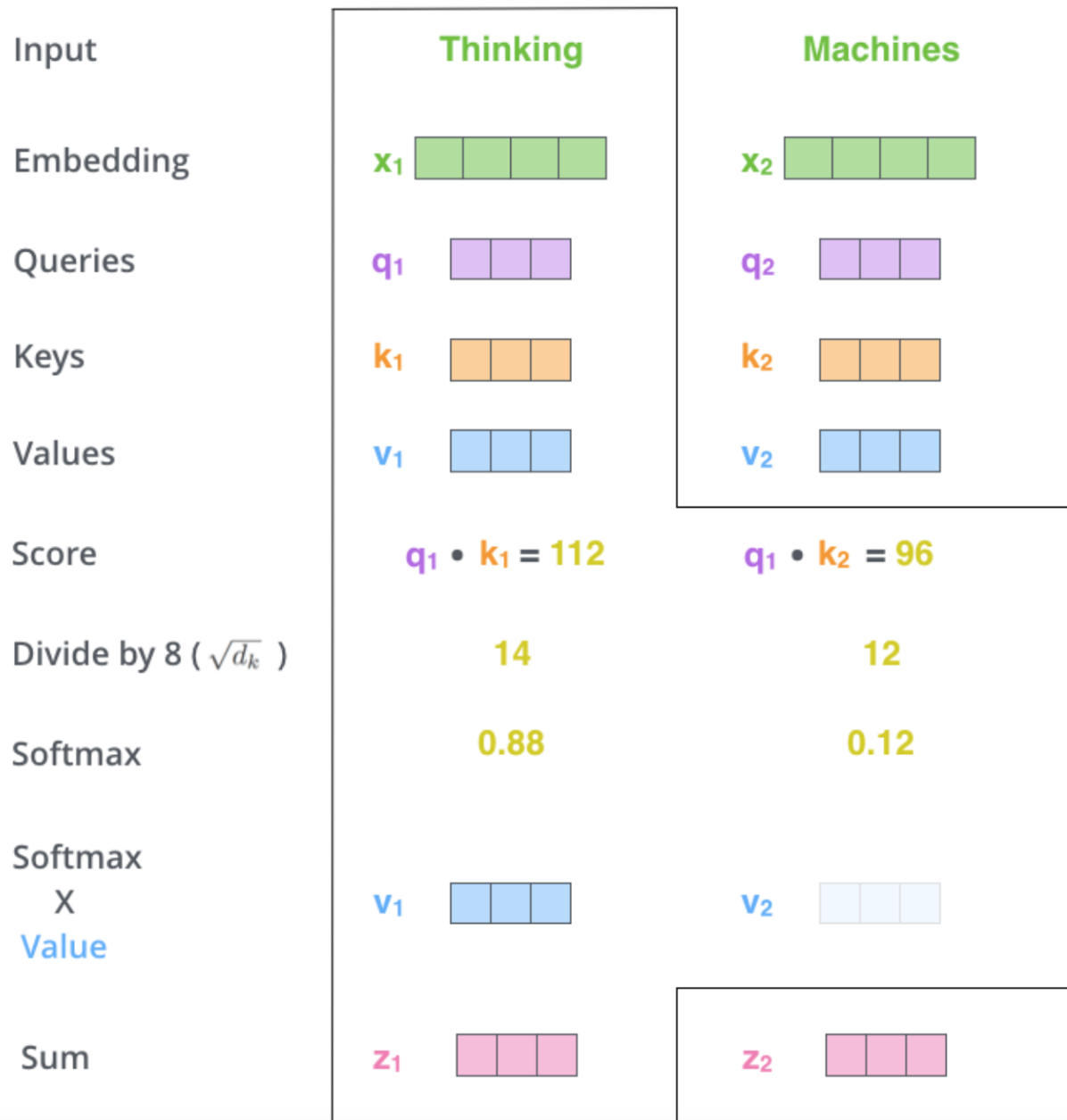
Values



W^V

Multiplying x_1 by the W^Q weight matrix produces q_1 , the "query" vector associated with that word. We end up creating a "query", a "key", and a "value" projection of each word in the input sentence.

d_k : the dimension
of the keys/values



The Illustrated Transformer

<https://jalammar.github.io/illustrated-transformer/>

[Jay Alammar](#)

Attention: Formal Version

Given an input vector series $(x_1, \dots, x_n) \in \mathcal{R}^d$, we may write it as a matrix $X \in \mathcal{R}^{d \times n}$. Given the attention's parameters $W^K, W^Q \in \mathcal{R}^{d_k \times d}$ and $W^V \in \mathcal{R}^{d_v \times d}$. Compute:

$$\begin{aligned} Q &= W^Q \cdot X = \begin{pmatrix} | & | & \cdots & | \\ q_1 & q_2 & \cdots & q_n \\ | & | & \cdots & | \end{pmatrix} \\ K &= W^K \cdot X = \begin{pmatrix} | & | & \cdots & | \\ k_1 & k_2 & \cdots & k_n \\ | & | & \cdots & | \end{pmatrix} \\ V &= W^V \cdot X = \begin{pmatrix} | & | & \cdots & | \\ v_1 & v_2 & \cdots & v_n \\ | & | & \cdots & | \end{pmatrix} \end{aligned}$$

A self-attention operation given Q, K, V returns a matrix $Z \in \mathcal{R}^{d_v \times n}$ of the following form:

$$Z = \begin{pmatrix} | & | & \cdots & | \\ z_1 & z_2 & \cdots & z_n \\ | & | & \cdots & | \end{pmatrix}$$

where

$$z_i = \sum_{j=1}^n sm \left(\frac{q_i \cdot k_j}{\sqrt{d_k}} \right) \cdot v_j$$

where sm is the softmax over $\left\{ \frac{q_i \cdot k_j}{\sqrt{d_k}} \right\}_{j=1}^n$

Multi-head Attention

- Transformer networks often use multi-head attention layers
- This means that a fixed number of attention mechanisms (“heads”)
 - Number of heads is usually one digit or low two digits
- Eventually, the output of the heads is concatenated to form the input to the next layer, and multiplied by an additional project matrix

Multi-head Attention

- Formally:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O$$

where $\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$

Residual Connections

- Residual connections effectively create a bypass around the module they go around (either a feed-forward or an attention)
- Assume $\text{SubLayer}(x)$ is the computation of the module, then after adding a residual connection, the layer becomes:

$$\text{Add\&Norm}(x) = \text{LayerNorm}(x + \text{SubLayer}(x))$$

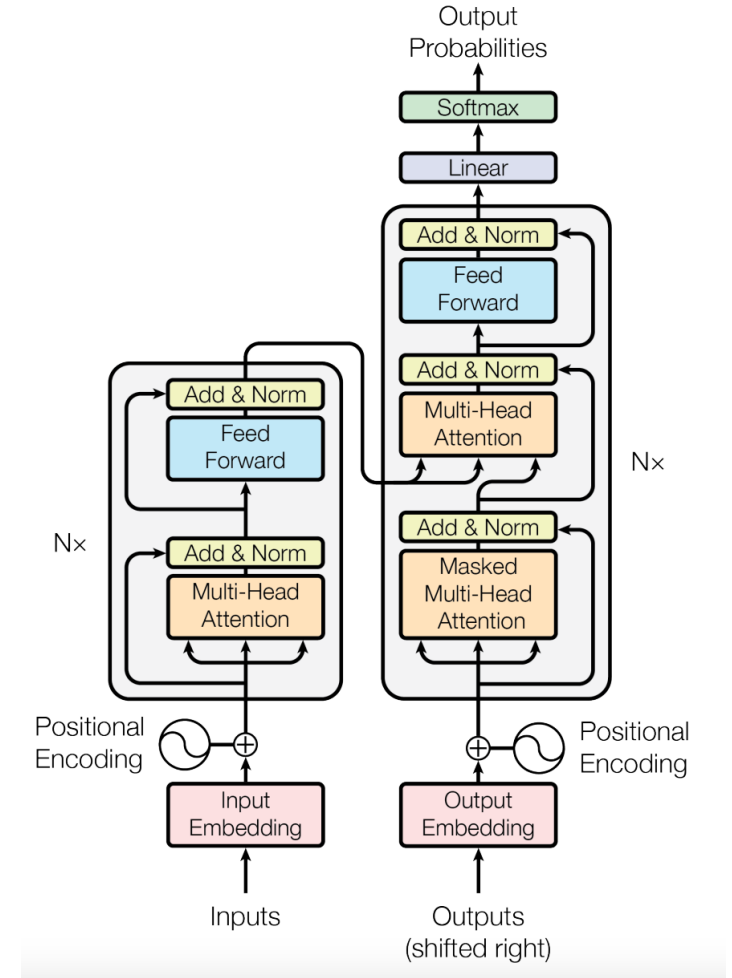
- LayerNorm is a mechanism for increasing the stability of the network (Ba et al., 2016)

The Transformer Model

- The Transformer (Vaswani et al. 2017) model is the standard neural model used in NLP today
 - Although it does have several important variants
- It is often used as a sequence-to-sequence model, but is not an RNN
 - This means that like the RNN encoder-decoder, it is essentially a model for predicting the next word, given the input sentence and prefix

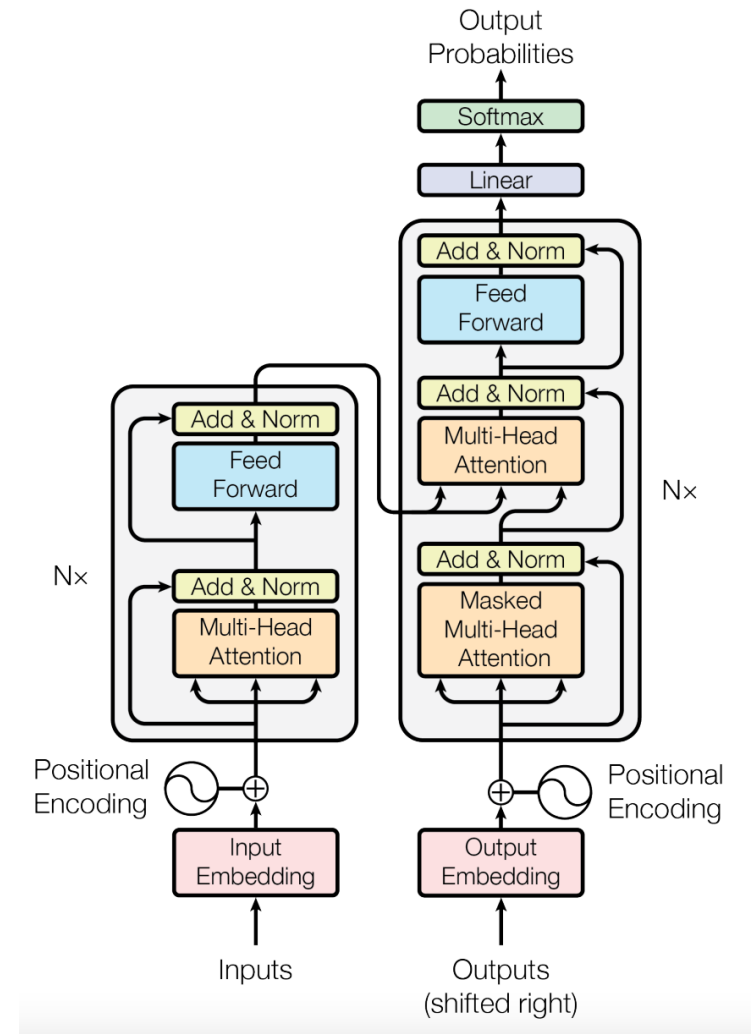
$$p_{\theta}(y_i | y_{i-1}, \dots, y_1, \mathbf{x})$$

- Unlike in RNNs, the input size is fixed, and special padding tokens are used to reach uniform input length



The Transformer Model

- The Transformer model is divided into blocks. Each block receives a representation of the tokens in the previous layer as vectors in and computes a representation of the next layer
- The network is both deep (often double-digit number of layers in the encoder and the decoder) and wide

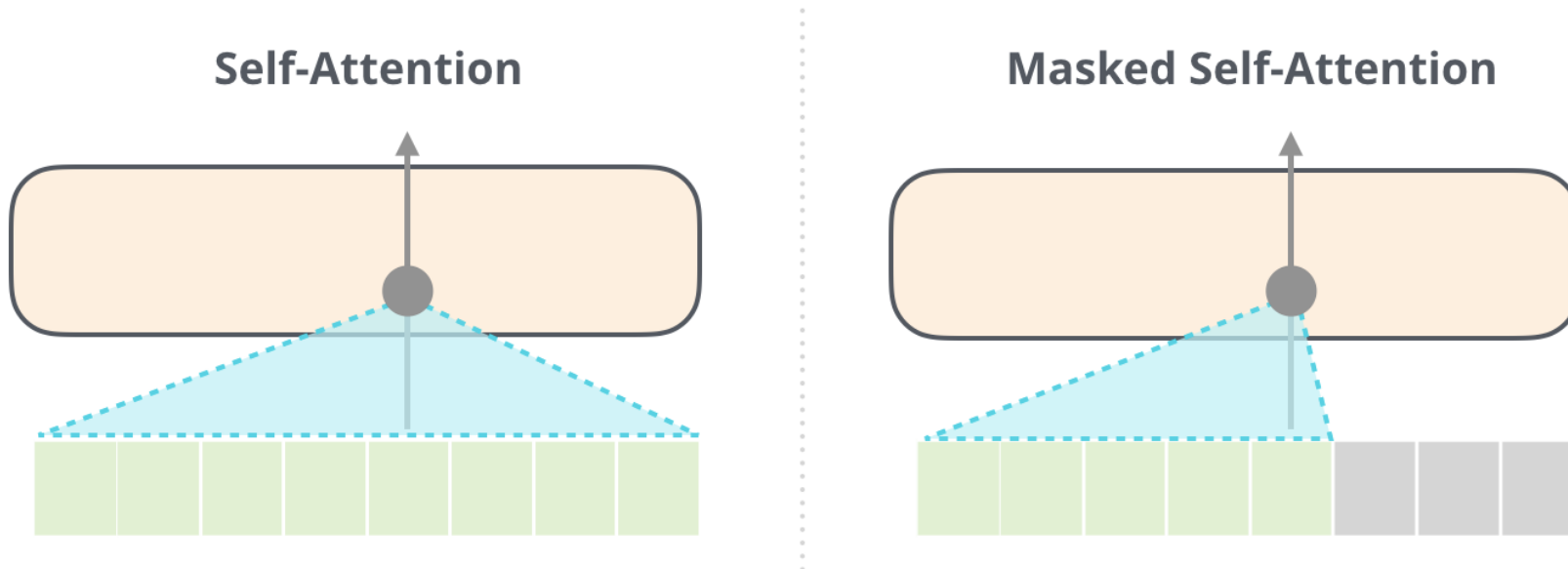


Decoder: Cross-Attention

- The decoder produces a distribution over the next word given a prefix
- To do so, it needs to attend both to the output of the encoder and to the encoding of the tokens in the prefix
- Cross-attention is the case where the queries Q come from one source and the keys K come from another
 - Otherwise, the attention mechanism stays the same
 - In the case of the transformer, queries come from the concatenation of the encoder's output and the output of the "masked multi-head attention"

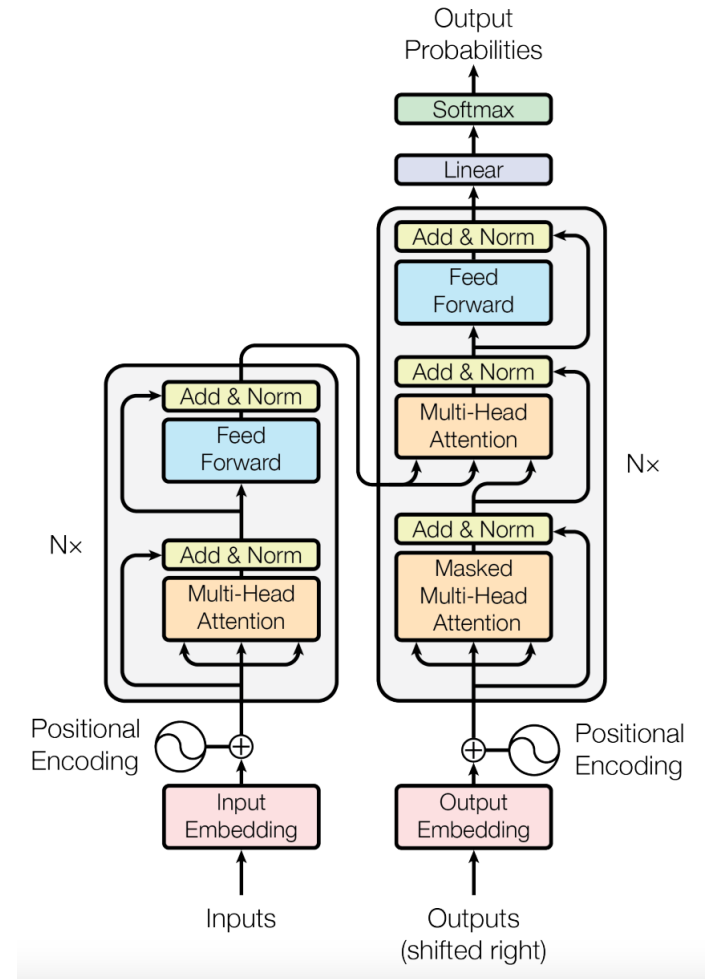
Decoder: Masked Attention

- In order to save compute, the decoder caches the previously computed activations
 - This means that the activations in position i may not depend on indices $j > i$



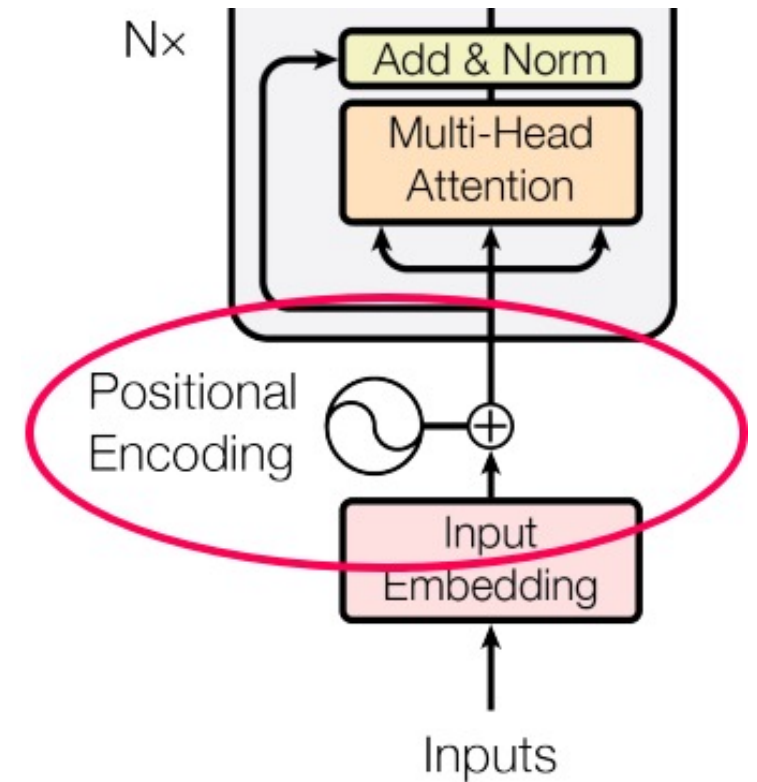
The Transformer Model

- The decoder for the model is similar in architecture to the encoder, but with several important differences:
 - Each decoder block includes a block of multi-head self-attention, followed by a block of attention over both the encoder input and the output of the self-attention
 - The input to the decoder is the prefix
 - The decoder outputs a probability over the possible next tokens



The Encoding of Positions in Transformers

- Tokens are represented in the Transformers as a concatenation of an input embedding (non-contextualized) and a positional embedding (different embedding for every position index)
- The positional embeddings are the only elements that break the symmetry between tokens
- Positional embeddings are randomly initialized and learned during training
- The model can therefore learn to take into consideration any patterns in the relative positions, as long as it is within the context window



Training and Decoding with the Transformer

- The Transformer is trained with standard log probability loss, similar to the ones used for neural language models

$$L(\theta) = \frac{1}{N} \sum_{i=1}^N \log[p_{\theta}(y_i | y_{i-1}, \dots, y_1, \mathbf{x})]$$

- Unlike RNNs, the Transformer does not require backpropogating through time
 - It is essentially a feed-forward network
 - Allows more parallelization in training
- The text is divided into segments using a sliding window (often with overlaps)
- Inference is essentially the same: can be done greedily, through beam search or using more advanced decoding techniques
 - Suffers from the same exposure bias...

Advantages of Transformers

- Transformers have shown superior results over RNNs in many NLP tasks, including Machine Translation
- Much easier to train than RNNs (no backpropagation through time)
- Less sensitivity to position
 - The input is given in parallel and not sequentially
 - Positional embeddings are learned

Machine Translation Evaluation

Machine Translation Evaluation

- Human subjective evaluation is the best but is time-consuming, expensive and inconsistent for long sentences.
- Automated evaluation comparing the output to **multiple human reference translations** is cheaper and generally correlates with human judgements.

The BLEU Score

- The most common measure is BLEU (Papineni et al., 2002)
- It determines number of n -grams of various sizes that the MT output shares with the reference translations
- Computes a modified precision measure of the n -grams in MT result
- Since precision-based measures favor short sentences, a penalty term is introduced for long translations

The BLEU Score

Cand 1: Mary no slap the witch green.

Cand 2: Mary did not give a smack to a green witch.

Ref 1: Mary did not slap the green witch.

Ref 2: Mary did not smack the green witch.

Ref 3: Mary did not hit a green sorceress.

Cand 1 Unigram Precision: 5/6

BLEU Score

Cand 1: Mary no slap the witch green.

Cand 2: Mary did not give a smack to the green witch.

Ref 1: Mary did not slap the green witch.

Ref 2: Mary did not smack the green witch.

Ref 3: Mary did not hit a green sorceress.

Cand 1 Bigram Precision: 1/5

BLEU Score

Cand 1: Mary no slap the witch green.

Cand 2: Mary did not give a smack to the green witch.

Ref 1: Mary did not slap the green witch.

Ref 2: Mary did not smack the green witch.

Ref 3: Mary did not hit a green sorceress.

Cand 2 Unigram Precision: 7/10

BLEU Score

Cand 1: Mary no slap the witch green.

Cand 2: Mary did not give a smack to the green witch.

Ref 1: Mary did not slap the green witch.

Ref 2: Mary did not smack the green witch.

Ref 3: Mary did not hit a green sorceress.

Cand 2 Bigram Precision: 4/9

Modified N -Gram Precision

- If we evaluate performance on a single candidate c against a set of references, then for each n -value ($n=1,\dots,N$):

$$p_n = \frac{\text{number of } n\text{-grams in } c \text{ that appear in at least one of the references}^*}{\text{number of } n\text{-grams in } c}$$

- We compute the geometric mean of n -gram precision over all n -grams up to size N (typically 4):

$$p = \left(\prod_{n=1}^N p_n \right)^{\frac{1}{N}}$$

* - if an n -gram appears multiple times in c , we do not count it more than the maximal number of times it appears in any of the references

Modified N -Gram Precision

- BLEU can be easily extended to evaluate many sets of candidates and references (a corpus)
 - We do not cover it here
- Returning to the example from before we get: (for $N=2$)

$$\text{Cand 1: } p = \sqrt[2]{\frac{5}{6} \frac{1}{5}} = 0.408$$

$$\text{Cand 2: } p = \sqrt[2]{\frac{7}{10} \frac{4}{9}} = 0.558$$

Brevity Penalty

- Not easy to compute recall to complement precision since there are multiple alternative gold-standard references and don't need to match all of them
- Instead, use a penalty for translations that are shorter than the reference translations
- Define effective reference length, r , for each sentence as the length of the reference sentence with the largest number of n -gram matches. Let c be the candidate sentence length:

$$BP = \begin{cases} 1 & \text{if } c > r \\ e^{(1-r/c)} & \text{if } c \leq r \end{cases}$$

BLEU Score

- Final BLEU Score: $BLEU = BP \times p$

Cand 1: Mary no slap the witch green.

Best Ref: Mary did not slap the green witch.

$$c = 6, \quad r = 7, \quad BP = e^{(1-7/6)} = 0.846$$

$$BLEU = 0.846 \times 0.408 = 0.345$$

Cand 2: Mary did not give a smack to a green witch.

Best Ref: Mary did not smack the green witch.

$$c = 10, \quad r = 7, \quad BP = 1$$

$$BLEU = 1 \times 0.558 = 0.558$$

The BLEU Score

- BLEU has been shown to correlate with human evaluation when comparing outputs from different SMT systems
- However, it does not correlate with human judgments when comparing SMT systems with human translations
- Other MT evaluation metrics have been proposed that claim to overcome some of the limitations of BLEU

Other Evaluation Methods

- Direct Assessment (Graham et al., 2015): a protocol for evaluation by crowd-sourcing
- We will discuss more nuanced semantic evaluation later in the course

Yvette Graham, Timothy Baldwin, Nitika Mathur.

"Accurate Evaluation of Segment-level Machine Translation Metrics", NAACL 2015